ENGINEER INTEGRATION GUIDE

LineFollowerPipeline

Pure-Java, zero-dependency line-following vision processor

Single-file library • Java 8+ • No external dependencies • Updated November 2026 (v3)

What's new in this version (v3)

- **Glare handling.** Saturated pixels (a lamp or window reflected by glossy tape or floor) are no longer read as line or floor. If glare hides the line in the scan strip it is followed from the line farther ahead; glare across the look-ahead is not read as a dead end; a line hidden mid-image is coasted on for up to 600 ms while the robot slows down (`glareMask`, `glareCoastMs`).
- **Corridor tracker.** The locked line is followed as a curve on the floor: 12 thin slices, each needing both tape edges, update a small Kalman filter. A highlight on the tape centre no longer splits the line; corners, dead ends and re-acquiring use the v2 logic (`corridorMode`). About 40 % less time per frame.
- **Distractor hardening.** Re-acquiring prefers the candidate that continues up the look-ahead; a line running into a dark mat is not continued along the mat's edge; soft-edged shadows count less (`edgePairCheck`).
- **Edge exits beat coasting.** A line leaving at the frame edge is pinned to ±50 at once, as in v2.
- **Richer callback.** `FrameData` adds `baseReconstructed`, `coasting`, `maskedFrac`, `corridorCoverage`, `corridorFitRms`, `curveA/B/C`.
- **No API break.** Nothing was removed or renamed; `corridorMode = false` + `glareMask = false` gives exactly v2.

Changelog — What an Integrator Needs to Know

In this guide NEW marks what v3 adds and CHANGED marks existing items whose behaviour changed in v3.

v3 — November 2026

CHANGE	WHAT IT MEANS FOR YOU	CODE CHANGE NEEDED?
Glare handling — new `glareMask`, `glareLumaMax`, `glareCoastMs`, `glareCoastMaxFrames`, `frameIntervalMs` (on by default)	Saturated pixels (a lamp or a window reflected by glossy tape or floor) are no longer read as line or floor. If glare hides the line in the scan strip, it is followed from the line farther ahead (`baseReconstructed`); glare rows in the look-ahead no longer read as a dead end; a line hidden in the middle of the image is **coasted on for up to 600 ms** (v2: held for 6 frames) while the robot slows down. See *Glare Robustness*.	No
Corridor tracker — new `corridorMode`, `corridorSlices`, `corridorWidthMult`, `minChainSlices`, `corridorMinConfidence`, `corridorProcessNoise` (on by default)	The locked line is followed as a curve on the floor: 12 thin slices along a corridor around the predicted curve, each needing **both tape edges**, update a small Kalman filter. A highlight on the tape centre no longer splits the line; when glare hides the near part, the line seen farther ahead still gives heading and curvature. Corners, dead ends, a line leaving the frame and re-acquiring use the v2 logic. About **40 % less time per frame**. See *Corridor Tracker*.	No
Distractor hardening — new `edgePairCheck` (on by default)	When the line is found again after a loss, the candidate that continues up the look-ahead wins (a short object at the straight-ahead column is no longer taken); a line running into a dark mat is not "continued" along the mat's edge (the robot slows down at the end as it should); a band without two sharp edges (soft shadow) counts less.	No

CHANGE	WHAT IT MEANS FOR YOU	CODE CHANGE NEEDED?
Edge exits beat coasting	A line that leaves at the frame edge — or that the look-ahead shows leaving through the edge — is pinned to that edge at once, as in v2. Only while glare hides the strip **and** the filter was following a curve toward that edge, the pin is capped at the filter's own steering for up to ~0.17 s, then becomes ±50.	No
New `FrameData` fields	`baseReconstructed`, `coasting`, `maskedFrac`, `corridorCoverage`, `corridorFitRms`, `curveA`, `curveB`, `curveC` — see *FrameData Fields*. `lineHeld` is also `true` while coasting.	No (optional for a debug HUD)
Test recorder — new `LineFollowerPipeline.Recorder` (off unless switched on)	A per-frame CSV log of the FrameData fields and a "capture" of the last ~2 s of frames that replays on the desktop. Works in any host app — it only uses the frame callback. See *Test Recording*.	No (optional, for tests)

Do I need to change my integration code? No. No public method, constant or field was removed or renamed — v3 only adds Config and FrameData fields. What changes is the **behaviour** under glare, around distractors and when the line is hidden (coast instead of a 6-frame hold). To get the exact v2 behaviour back (checked frame by frame against v2), set:

```
LineFollowerPipeline.Config v2 = new LineFollowerPipeline.Config();
v2.corridorMode = false;   // v2 tracking (also turns off the v3 distractor hardening)
v2.glareMask    = false;   // no glare mask / look-ahead base / coast: the v2 6-frame hold
pipeline.configure(v2);
```

v2 — October 2026

CHANGE	WHAT IT MEANS FOR YOU	CODE CHANGE NEEDED?
Distractor rejection — new `rejectDistractors`, `bandMaxWidthMult`, `bandMinWidthMult` (on by default)	A dark object next to the track (a black chair leg, a bag, a shadow, the dark floor past a table edge) is no longer taken for the line. A band only counts as the line if its **width** and **line-ness** match the line measured at calibration. See *Distractor Rejection*.	No
Steering uses the whole ⊥ — new `steerOffsetWeight`, `steerHeadingWeight`, `steerLookAhead`, `steerHeadingMinRows`, `steerHeadingCalibMaxDeg`	Steering = **offset** of the line in the scan strip (as before) **+ heading** of the line from the look-ahead (new). A robot at an angle to the line now gets a steering value even when the line crosses the strip at the calibrated column (±15° → about ±8, ±30° → about ±17). Range and sign are unchanged. See *Steering from the Whole* ⊥.	No — but the steering values differ when the robot is at an angle, so re-check your steering gain
Hidden line is held — new `trackHoldFrames`	If the line disappears in the **middle** of the image (glare, a shadow, an object over it), the last steering and tracking are held for up to 6 frames instead of jumping to ±50 at once. During the hold `trackFound` is `false` and `lineHeld` is `true`. A line that leaves at the frame **edge** is still pinned to ±50 at once.	Only if your code reacts to `trackFound == false` on the very first frame

CHANGE	WHAT IT MEANS FOR YOU	CODE CHANGE NEEDED?
Line lock follows the predicted position	The lock gate is centred on where the line is expected (last position + recent motion) and can no longer grow to the whole frame. The lock correction (`trackRelockFrames`) needs 3× as many frames to leave a valid-looking line.	No
Calibration learns more	Besides the colours and the straight-ahead column, calibration now records the line **width**, how line-like its pixels are, and the **heading zero** (phone yaw on its mount). **Calibrate on a straight section, robot straight on the line.**	No
New `FrameData` fields	`steerOffsetTerm` , `steerHeadingTerm` , `headingDeg` , `headingConfidence` , `lineHeld` , `rejectedBands` — see *FrameData Fields*.	No (optional for a debug HUD)

In v2, steering started to react to the robot's heading, distractors were ignored, and a line hidden mid-image was held briefly. To get the exact v1 behaviour back, set:

```
LineFollowerPipeline.Config v1 = new LineFollowerPipeline.Config();
v1.corridorMode       = false;   // no v3 corridor tracker
v1.glareMask          = false;   // no v3 glare handling
v1.rejectDistractors  = false;   // original band selection (also turns off the hold)
v1.steerHeadingWeight = 0f;      // steering = offset only, exactly as in v1
pipeline.configure(v1);
```

Earlier changes

- **11 Oct 2026** — **Steering output is** `-50 … +50` (was `-100 … +100`): only the output is halved, the pipeline itself is unchanged. If your control program multiplies steering by a gain, **double the gain** to keep the same steering authority. `trackingMinValue` **default is** `5` (was `2`), so Tracking is `5 … 100` .
- **8 Oct 2026** — Automatic colour learning at calibration (`autoColor = true` by default; `trackIsBright` is now only the manual fallback); YUV 4:2:0 and ARGB input; Tracking ramps down with the remaining visible road; lock correction after turns (`trackRelockFrames`); `minContrast` ; richer `FrameData` (calibration phase / progress / problem, learned colours, threshold, line width, frame size).

What This Library Does

`LineFollowerPipeline.java` is a pure-Java, zero-dependency line-following vision processor. You feed it raw camera frames — **grayscale, YUV 4:2:0 or ARGB** — and it returns two integer control values your robot can act on directly: a **steering direction** and a **speed/tracking signal**. No OpenCV, no Android APIs, and no native libraries are required.

Requirements

- Java 8 or higher
- One file only: `LineFollowerPipeline.java`
- No external dependencies

Minimal Integration (4 Steps)

1. **Create one instance (keep it alive for the session)**

```
LineFollowerPipeline pipeline = new LineFollowerPipeline();
```

2 Calibrate — feed frames until it returns `true`

```
while (!pipeline.calibrate(grayscaleBytes, width, height)) {
    grayscaleBytes = camera.getGrayscaleFrame();
}
```

3 Call `processFrame()` on every camera frame

```
int[] result = pipeline.processFrame(grayscaleBytes, width, height);
```

4 Read the two output values

```
int steering = result[0];  // use for direction control
int tracking = result[1];  // use for speed control
```

Have colour frames? Use `calibrateYuv420(y, u, v, w, h)` / `processFrameYuv420(…)` (Android camera) or `calibrateArgb(…)` / `processFrameArgb(…)` instead — colour input separates even line/floor colours of the same brightness. See *Input Formats*.

Calibration is required. `processFrame()` must not be called until `calibrate()` has returned `true`. The robot must be correctly placed on the line — straight, on a straight section, with floor visible on both sides — while calibrating. See *Calibration* below.

Input Formats

Each format has a matching calibrate / process pair. Use the **same format** for both.

FORMAT	CALIBRATE	PROCESS	DATA
Grayscale	`calibrate(gray, w, h)`	`processFrame(gray, w, h)`	`byte[] gray` — `w × h` luma bytes
YUV 4:2:0 *(recommended on Android)*	`calibrateYuv420(y, u, v, w, h)`	`processFrameYuv420(y, u, v, w, h)`	`y` — `w × h` bytes; `u`, `v` — `((w+1)/2) × ((h+1)/2)` bytes each
ARGB	`calibrateArgb(argb, w, h)`	`processFrameArgb(argb, w, h)`	`int[] argb` — `w × h` packed `0xAARRGGBB`

Important: all arrays must be packed with **no row padding** (and the U/V planes with no pixel interleaving). Android CameraX planes are padded / interleaved — strip that before passing the data in (see `copyLumaPlane` / `copyChromaPlane` in RoboPhoneApp's `MainActivity`). Arrays that are too short throw `IllegalArgumentException`.

Which format should I use?

- **Colour (YUV / ARGB)** separates any two colours that look different — even two colours of the **same brightness** (e.g. red on green).
- **Grayscale** works for any two colours that differ in **brightness** (white on black, black on blue, white on brown, black on white). Calibration reports `ISSUE_LOW_CONTRAST` and keeps retrying if the two colours have the same brightness.

Output Format

`processFrame()` returns a **reused** `int[2]` array. Copy the values immediately if you need to keep them — the same array is overwritten on the next call.

INDEX	NAME	RANGE	MEANING
`result[0]`	**Steering**	-50 … +50	`0` = robot is on the line and parallel to it. **Negative = turn left** (the line is to the left, or heads to the left). **Positive = turn right** (the line is to the right, or heads to the right). It combines where the line crosses the scan strip (offset) and which way the line runs ahead (heading) — see *Steering from the Whole* ⊥. `±50` toward the side where the line was lost (see *When the Line Is Lost or Hidden*).
`result[1]`	**Tracking**	5 … 100	`100` = straight path ahead, full speed. Drops as a turn or curve is detected, and ramps down linearly as the visible road ahead gets shorter (dead end, sharp corner). Never falls below `Config.trackingMinValue` (default **5**), so the robot is never sent a full stop. Set that field to `0` for the full 0–100 range.

Typical robot wiring:

```
robot.setSteeringAngle(result[0]);   // direction
robot.setSpeed(result[1]);           // speed — naturally slows before turns
```

Optional Configuration

To tune the pipeline for your camera mount, track, or robot body size, create a `Config` object and pass it **before the first frame**:

```
LineFollowerPipeline.Config cfg = new LineFollowerPipeline.Config();
// --- apply your settings here ---
pipeline.configure(cfg);  // must be called BEFORE the first processFrame()
```

Calling `configure()` later discards the calibration (colours given with `setTrackColors()` are kept), so calibrate again afterwards. The **Range** column gives the values the code accepts or that make sense; values outside it are clamped or have no useful effect.

Line Colour

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`autoColor`	boolean	true	true / false	**Normal setup.** Learn the line and floor colours during calibration — any two different colours work and `trackIsBright` is ignored. `false` = classic grayscale mode using `trackIsBright`.
`trackIsBright`	boolean	true	true / false	**Manual fallback only** — not needed with `autoColor = true`. Used when `autoColor = false`, and before the colours have been learned (e.g. if `processFrame()` is called before calibration has finished). `true` = bright line on dark floor. `false` = dark line on bright floor.
`colorCalibFrames`	int	5	≥ 1	Frames sampled to learn the two colours.
`colorMinSeparation`	float	4.0f	≥ 0	Minimum line-vs-floor colour separation (in standard deviations of the colour noise). Below this, calibration discards the samples and retries.
`colorMinDistance`	float	12f	≥ 0	Minimum distance between the two learned colours (YUV grey-levels). Rejects "two colours" that are only sensor noise.
`colorMinThreshold`	int	112	0 … 255 (useful 64 … 192)	In colour mode the floor maps to 64 and the line to 192 internally; the threshold never drops below this so noisy floor pixels are never taken for the line.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`thresholdMargin`	int	0	grey-levels, ± (threshold kept in 15 … 240)	Bias added to the auto threshold. Increase if false positives occur. Decrease if the line is being missed.
`minContrast`	int	24	0 … 255	If the two halves of the Otsu split differ by less than this many grey-levels, the strip is a plain surface with **no line** (prevents "finding" a line in sensor noise). `0` disables.
`bottomIgnoreFrac`	float	0.15f	0 … 0.9	Fraction of the frame bottom to ignore — use this to exclude the robot's own body from the image. Increase until the chassis disappears from the scan. (RoboPhoneApp uses `0.25`.)

Scan Strip (H-ROI — Near Line / Steering)

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`hRoiCenterYFrac`	float	0.85f	0 … 1	Vertical position of the near scan strip as a fraction of frame height. `0.85` = 85% down from the top (near the bottom). The strip is moved up if it would overlap the ignored robot-body band.
`hRoiThickPx`	int	24	1 … frame height	Height of the scan strip in pixels.

Look-Ahead Column (V-ROI — Turn Preview / Tracking)

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`vRoiHalfWidthPx`	int	80	≥ 1 px	Half-width in pixels of the look-ahead column. Increase on tight curves so the line stays inside the scan window. With `rejectDistractors` the window is automatically widened on rows where the line itself (plus a small margin) would not fit — e.g. wide tape at 960 × 1280.
`vRoiHeightFrac`	float	0.5f	0 … 1	How far ahead to look, as a fraction of frame height. `0.5` = half a frame ahead. Larger = more warning of far turns, but noisier.
`vRoiAdaptiveThreshold`	boolean	true	true / false	Gives the look-ahead its own Otsu threshold. **Recommended** — the far line is dimmer than the near line and benefits from separate calibration.

Camera Mount

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`cameraTiltDeg`	float	45f	0 … 90 (degrees)	How far the phone's camera points below horizontal. Used for the perspective taper of the look-ahead, the look-ahead zero reference, the road-length slowdown, the line-width test of the look-ahead and the **heading** used by the steering (the line is projected onto the floor). Measure it on the real robot — a wrong value scales the heading part of the steering. Keep it large enough that the whole look-ahead lies below the horizon (floor only).
`verticalFovDeg`	float	60f	1 … 179 (degrees; phones ~55–65)	Vertical field of view of the camera (along the frame height). Most phone back cameras are ~55–65°. Used together with `cameraTiltDeg` for the floor projection.

Track Memory

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`trackMemory`	boolean	true	true / false	Lock onto the line and only search a gate around its predicted position (last position + recent motion), so reflections elsewhere are ignored. `false` = search the whole width every frame.
`trackMemoryGateMult`	float	3.0f	> 0	Gate half-width as a multiple of the measured line width (with `rejectDistractors`, never more than an acceptable band width, so the gate cannot grow to the whole frame). The gate is never wider than half the frame.
`trackMemoryMinGatePx`	int	48	≥ 0 px	Minimum gate width in pixels regardless of line width.
`trackLostGraceFrames`	int	8	≥ 0 frames	Frames to wait after losing the line before searching the full frame for a new one. At 30 fps, `8` ≈ 0.25 seconds.
`trackRelockFrames`	int	6	≥ 0 frames	Lock correction. If a band that looks much more like the line (near the calibrated straight-ahead column, with the calibrated line width) is seen outside the lock for this many frames in a row, the lock moves to it — so a reflection or neighbouring tape grabbed after a turn cannot keep steering offset. With `rejectDistractors`, a locked line that still looks valid is only abandoned after **3×** this many frames. `0` disables.

When searching for the line (start-up, or after it was lost), the pipeline no longer simply takes the biggest band in view: it prefers the band nearest the calibrated straight-ahead column whose width matches the line width measured during calibration. While the locked line is lost at the frame edge, steering reads ±50 toward the side it left; a line that vanishes mid-image is held first (see *When the Line Is Lost or Hidden*).

Distractor Rejection (chair, bag, shadow, table edge)

Objects with a colour close to the line's (a black chair leg on a black-tape track, a shadow on wood, the dark floor beyond a table edge) pass the threshold exactly like the line. With `rejectDistractors` on, a band only counts as the line if it **looks like the line learned at calibration**:

- **Width** — the band must be between `bandMinWidthMult` × and `bandMaxWidthMult` × the calibrated line width (the maximum is corrected for the line's slant in the image). Wide blobs, dark regions at the frame edge and thin specks are rejected.
- **Line-ness** — among acceptable bands, the one whose pixels are as line-like as the calibrated line wins (a shadow on wood only just passes the threshold; the tape is well past it).
- **Continuity** — while locked, the band nearest the *predicted* position wins (on the scale of one line width), and a band that is the line **merged** with a blob or shadow is cut back to the part that continues the line (its strong core, or the edge that sits where the line's edge was).
- **Look-ahead** — the row-by-row tracker of the V-ROI applies the same width test on every row (perspective-scaled), so it cannot step onto a nearby object; a run cut by the scan window is only used through the edge that continues the line.
- **Hidden line** — if the locked line disappears in the middle of the image (glare, a shadow or an object over it) it is coasted on for `glareCoastMs` (v3; with `glareMask = false` the last steering and tracking are held for `trackHoldFrames`, as in v2); a line that leaves at the frame edge (a real turn) is still pinned to that edge at once.
- **Relock** — the lock correction (`trackRelockFrames`) still exists, but a valid-looking locked line is only abandoned after three times as many frames, so a chair leg near the centre cannot steal the lock during a turn.

What you must ensure — the rules compare everything with the line measured at calibration, so:

- **Calibrate with the robot on the line**, straight, on a straight section, with floor visible on both sides, and **no distractor inside the scan strip** during calibration.
- **Use one tape width for the whole track** (a reasonable width: the line should cover well under half of the scan strip). Width changes of more than about 2× (`bandMaxWidthMult`) or less than 0.3× (`bandMinWidthMult`) are treated as "not the line". **Recalibrate** when you change tape.
- Set `cameraTiltDeg` / `verticalFovDeg` close to the real camera — the look-ahead width test uses them.
- Remaining limit: a dark object of **exactly the tape's width**, lying where the line is predicted to be, can still be taken for the line while it is in the scan strip (v3: no longer when the line is re-acquired — the real line continues up the look-ahead, the object does not).

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`rejectDistractors`	boolean	true	true / false	Turn the rules above on. `false` = the original (v1) band selection (this also turns off the hold and the predicted gate).
`bandMaxWidthMult`	float	2.0f	≥ 1.3 (useful 1.5 … 3)	Widest band accepted as the line, as a multiple of the expected (calibrated, slant-corrected) width. A real line stays within about ±30 %; a line merged with a blob is ≥ 2.5×.
`bandMinWidthMult`	float	0.3f	0 … 1 (0 = no minimum)	Narrowest band accepted (fraction of the calibrated width); bands cut by the frame edge are exempt.
`trackHoldFrames` CHANGED	int	6	0 … trackLostGraceFrames	Frames the last output is held when the line vanishes mid-image (6 ≈ 0.1 s at 60 Hz). `0` = report the loss at once (v1). During the hold `trackFound` is `false` and `FrameData.lineHeld` is `true`. Only active with `rejectDistractors = true` and `glareMask = false` — with the v3 default `glareMask = true` the line is coasted on for `glareCoastMs` instead.

Steering from the Whole ⊥ (offset + heading)

The pipeline looks at the line as a ⊥ (an inverted "T"): the **base** is the horizontal scan strip (H-ROI), the **vertical part** is the line followed up the look-ahead (V-ROI). The v1 steering used only the base of the ⊥ — where the line crosses the scan strip — so a robot at 30° to the line and a robot straight on it read the **same** value if the line crossed the strip at the same point. The steering now also uses the line's **heading** from the vertical part of the ⊥:

```
offset   = (centroid − calibrated column) / (W/2) × 100                    (v1 steering)
heading  = 100 × L × tan(ψ − ψ₀) / (floor half-width seen by the H-ROI)    (ψ − ψ₀ limited to ±75°)
L        = steerLookAhead × (floor distance from the camera to the H-ROI)
steering (internal) = clamp(steerOffsetWeight × offset + steerHeadingWeight × c × heading, −100, +100)
result[0] = round(steering / 2)                                            → [-50 … +50]
```

- **W** is the frame width in pixels; the **centroid** is where the line crosses the scan strip; the **calibrated column** is the straight-ahead column measured during calibration (`FrameData.steerRefXFrac`).
- **ψ** is the direction of the line on the floor where it crosses the scan strip: the H-ROI centroid and the look-ahead rows on the line are projected onto the floor with the camera geometry (`cameraTiltDeg`, `verticalFovDeg`, exact pinhole model) and fitted with a parabola; its tangent at the strip is ψ. Positive = the line heads to the right.
- **ψ_0** is the heading measured during calibration (phone yaw on its mount), so a robot correctly placed on a straight line still reads `0`. Calibrate on a straight section; a calibration heading larger than `steerHeadingCalibMaxDeg` is ignored ($\psi_0 = 0$).
- Together the two parts are the lateral position of a **pure-pursuit target point** a distance L further along the line, in the same units as the v1 steering.

- **c** (the heading confidence, `FrameData.headingConfidence`, 0 … 1) fades the heading out when too little of the line is visible ahead (end of line, line leaving the frame) or when the line is not a smooth curve (a 90° corner, a junction) — the steering then falls back to the v1 offset only.
- **Signs**: both parts are positive when the robot must turn **right** (line to the right / line heading right). `FrameData.steerOffsetTerm` and `steerHeadingTerm` are the two weighted parts in output units (already halved), so before clamping and rounding `steering ≈ steerOffsetTerm + steerHeadingTerm`.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`steerOffsetWeight`	float	1.0f	≥ 0 (keep 1)	Weight of the offset (v1) part.
`steerHeadingWeight`	float	1.0f	≥ 0	Weight of the heading part. `0` **= exactly the v1 steering.**
`steerLookAhead`	float	0.5f	> 0 (tested 0.25 … 1.0)	L as a multiple of the camera-to-strip floor distance. Larger = the heading counts more: a robot at an angle to the line straightens out sooner, but it runs a little more on the inside of tight curves and, with a high steering gain plus delay, can start to weave. `0.5` was the best balance in the closed-loop simulation. With the defaults, a robot at ±15° whose line crosses the strip at the calibrated column reads about ±8, at ±30° about ±17.
`steerHeadingMinRows`	int	24	≥ 1 rows	Look-ahead rows on the line needed for the heading at full weight.
`steerHeadingCalibMaxDeg`	float	10f	≥ 0 (degrees)	Largest heading accepted as the calibration zero.

Glare Robustness (v3) NEW

Strong light — a lamp or a window reflected by glossy tape (most often) or a glossy floor — saturates pixels: they lose their colour and can be read neither as line nor as floor. With `glareMask` on:

- **Masked pixels** (brightness `Y ≥ glareLumaMax`; ARGB: any channel ≥ 250) are left out of every threshold histogram. Inside a band they count as part of it, so a highlight on the tape centre does not split the line.
- **Base hidden by glare** — if the locked line is not visible in the scan strip, it is followed from the look-ahead and its position in the strip is taken from the line farther ahead (`FrameData.baseReconstructed`). A sharp curve seen only from afar is ambiguous: then the pipeline coasts rather than guess.
- **Glare rows are not the end of the road** — look-ahead rows where the line is hidden by glare are skipped, so Tracking does not drop as if the line ended.
- **Coasting** — a line hidden completely mid-image is coasted on for up to `glareCoastMs`: its position continues its last motion, the heading part fades out and Tracking ramps down (never faster than before, slower for a sharp curve seen ahead). The time comes from `System.nanoTime()` (or `frameIntervalMs`), at most 100 ms per frame, and the coast also ends after `glareCoastMaxFrames`.
- **Edge exits are not coasted** and each eighth of the scan strip gets its own Otsu threshold.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`glareMask` NEW	boolean	true	true / false	Turn the glare handling above on. `false` (with `corridorMode = false`) = exactly v2, including the `trackHoldFrames` hold.
`glareLumaMax` NEW	int	245	1 … 255	Brightness at or above which a pixel is masked as glare (gray / YUV input). ARGB input: any channel ≥ 250.
`glareCoastMs` NEW	float	600f	≥ 0 ms (0 = no coast)	Longest coast while the line is hidden mid-image. At 0.6 m/s, 600 ms ≈ 36 cm of blind driving.
`glareCoastMaxFrames` NEW	int	36	≥ 1 frames	Backup limit: the coast also ends after this many frames (36 = 600 ms at 60 fps).

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`frameIntervalMs` NEW	float	0f	≥ 0 ms (0 = measure)	Time between frames for the coast clock. `0` = measured with `System.nanoTime()`; set it for repeatable results (tests, simulation, replay). Each frame counts at most 100 ms.

Corridor Tracker (v3) NEW

When the line is locked, the pipeline follows it as a curve on the floor, `x = A + B·u + C·u²` (`u` = distance beyond the scan strip, `x` = sideways, camera heights):

- **Slices** — `corridorSlices` thin slices, evenly spaced in floor distance up to the far end of the look-ahead, each searched in a corridor (`corridorWidthMult` × the perspective-scaled line width) around the predicted curve.
- **Edge pairs** — in each slice the line is a rising *and* a falling edge at the expected width; glare columns are filled from their neighbours, so a highlight on the tape centre is bridged.
- **Kalman filter** — the near slices update a small filter of the curve; steering uses the v2 formula with `A` as the offset and `B` as the heading. When glare hides the near part, the far slices update heading and curvature and the near position comes from the prediction.
- **Hand-over to v2** — corners, junctions, dead ends, a line leaving the frame, jumps, too few slices, low confidence (`corridorMinConfidence`), relocks and re-acquiring use the v2 logic; `FrameData.corridorCoverage` is then 0.
- **Distractor hardening** — a band cut out of a wider one counts only if the line continues up the look-ahead; the edge of a mass of the line's colour (a mat) is not counted as road (bounded: if the line is seen past a blob, Tracking stays ≥ 60); re-acquiring prefers the candidate that continues farthest; with `edgePairCheck` a band without two sharp edges counts a third.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`corridorMode` NEW	boolean	true	true / false	Turn the corridor tracker (and the v3 distractor hardening) on. `false` = v2 tracking.
`corridorSlices` NEW	int	12	4 … 32	Number of slices from the scan strip to the far end of the look-ahead.
`corridorWidthMult` NEW	float	2.2f	≥ 1.2 (useful 1.8 … 3)	Corridor width per slice as a multiple of the expected line width (never narrower than the line plus a margin).
`minChainSlices` NEW	int	4	≥ 2	Slices on the line needed (when the near part is visible) to use the corridor result.
`corridorMinConfidence` NEW	float	0.35f	0 … 1	Below this confidence (coverage × fit quality) the frame uses the v2 logic.
`corridorProcessNoise` NEW	float	1.0f	> 0	How fast the curve may change between frames in the filter. Larger = follows sudden changes sooner, smooths less.
`edgePairCheck` NEW	boolean	true	true / false	With `corridorMode`: a scan-strip band without both sharp edges counts only a third.

Sensitivity & Tracking

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`minWeightFrac`	float	0.01f	0 … 1	Minimum fraction of scan strip pixels that must be "track" to trust the detection. Raise to reject weak signals.
`trackOffsetFullFrac`	float	1.0f	> 0 (usually 0.5 … 1)	How far the look-ahead line must lean (as a fraction of half-frame width) to push Tracking to its minimum. Lower (e.g. `0.6`) makes turns read earlier.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`trackLengthSlowdown`	boolean	true	true / false	Also limit Tracking by how much road is still visible ahead (floor distance, perspective-corrected). Approaching a dead end or a sharp corner, Tracking ramps down **linearly with the remaining road** instead of dropping from 100 to the minimum in one frame. A fully visible straight road still reads 100.
`trackLengthFullFrac`	float	0.8f	0 … 1	Fraction of the look-ahead distance the line must reach for the road-length part to read 100. Keeps the dim far edge from lowering Tracking on straight roads.
`trackingMinValue`	int	5	0 … 100	Floor for the Tracking output. `result[1]` will never go below this value, so the robot never receives a full stop signal. Set to `0` to allow the full 0–100 range.
`curveFullScaleDeg`	float	30f	—	**Not used** by the current code (no effect). Kept so existing code that sets it still compiles.

Tracking Minimum Value

By default Tracking **never reaches zero**: `result[1]` is clamped to a floor of 5, so the robot always receives at least a small speed signal. This prevents mechanical problems caused by a complete stop command.

VALUE	EFFECT
5 *(default)*	Tracking minimum is 5 — the robot always gets at least a small speed signal.
10	Raise the floor further.
0	Disable the floor — allow the full 0–100 range.

```
cfg.trackingMinValue = 5;   // set before calling pipeline.configure(cfg)
```

Calibration Frames

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`calibFrames`	int	10	≥ 1	Number of solid detections `calibrate()` collects before locking the straight-ahead zero reference (it takes their median), the line width, the line-ness level and the heading zero. Assumes the robot is correctly placed on the line. Increase for more robust calibration.

Calibration

Before calling `processFrame()` you must calibrate. Place the robot correctly on the line, then call `calibrate()` (or the YUV / ARGB version) once per camera frame until it returns `true`:

```
// Phase 1 — Calibration (at startup, or when the user presses calibrate)
while (!pipeline.calibrate(grayscaleBytes, width, height)) {
    // keep feeding frames until calibration is complete
    grayscaleBytes = camera.getGrayscaleFrame();
}

// Phase 2 — Normal operation
int[] result = pipeline.processFrame(grayscaleBytes, width, height);
```

Calibration has two phases:

1. **Learn the colours** (`colorCalibFrames`, default 5 frames). The scan strip is sampled and split into two colour clusters (k-means). The **floor** is the cluster that fills both the left and right frame edges; the other is the **line**. If that fails — typically a shadow or uneven light making part of the floor almost as dark as a dark line — 3 and then 4 clusters are tried: the line is the cluster that forms one band and never reaches the frame edges, and all other clusters (lit floor, shadowed floor…) count as floor. A Fisher linear discriminant then gives the colour direction that best separates line from floor, and every later frame is converted to a "line-ness" image (line bright, floor dark) before the usual Otsu + line-lock processing. Skipped when `autoColor = false` or when the colours were given with `setTrackColors()`.
2. **Find straight-ahead** CHANGED Collects `calibFrames` (default 10) solid line detections — preferring the band where colour learning saw the line and, with `rejectDistractors`, of the width colour learning measured — and takes their **median** as: the steering zero reference (straight-ahead column); the typical **line width** and **line-ness level** (used by distractor rejection); and the **heading zero** ψ_0 (only when `steerHeadingWeight ≠ 0`): the median line heading of the frames that saw enough look-ahead. It is used only if at least half of the frames saw it and it is within `steerHeadingCalibMaxDeg`; otherwise $\psi_0 = 0$.

With the defaults calibration needs at least **15 frames** (5 + 10). It returns `true` once both phases are done. Progress and problems are reported in the callback (`calibPhase`, `calibProgress`, `calibIssue`):

CALIBISSUE	MEANING	WHAT TO DO
`ISSUE_NONE`	All good	—
`ISSUE_LOW_CONTRAST`	The two colours in view are too similar, or there is only one surface	Make sure the line is in the STEER strip; for same-brightness colours use YUV/ARGB input; even out the lighting
`ISSUE_NO_FLOOR`	No colour fills both frame edges	Place the robot **on** the line, with floor visible on both sides
`ISSUE_NO_LINE`	Colours learned, but no line band found this frame	Check the line is inside the STEER strip

Before you calibrate — checklist

- The robot stands **on the line, straight**, on a **straight section** (the heading zero is learned here).
- The line is inside the scan strip, near the middle, with **floor visible on both sides** (both frame edges must show floor).
- No chair leg, bag, shadow edge or second tape inside the scan strip.
- The tape width is the one used on the whole track.

If calibration cannot complete, `calibrate()` simply keeps returning `false` — there is no timeout. Show `calibIssue` to the user, fix the cause (see the table) and keep feeding frames; call `requestRecalibration()` to start again from phase 1. Do **not** drive on `processFrame()` output before calibration is done: until then the steering zero is the image centre, there is no calibrated line width (so distractor rejection cannot work), and if the colours were not learned yet the frame is read as plain brightness with `trackIsBright` polarity.

Giving the colours yourself (optional)

```
pipeline.setTrackColors(0xFF000000 /* black line */, 0xFF1E3CC8 /* blue floor */);
pipeline.clearTrackColors();   // go back to learning them
```

`setTrackColors()` has no effect when `autoColor = false`, and throws `IllegalArgumentException` if the two colours are the same.

Re-calibrating mid-run

```
pipeline.requestRecalibration();
// then feed frames to calibrate() again before resuming processFrame()
```

Recalibrate whenever you move to a track with different colours, very different lighting or a different tape width.

When the Line Is Lost or Hidden CHANGED

SITUATION	STEERING RESULT[0]	TRACKING RESULT[1]	TRACKFOUND	LINEHELD
Line visible	offset + heading	from the look-ahead	`true`	`false`
Near line hidden by glare, line visible farther ahead (`baseReconstructed`) NEW	from the line ahead (its position in the strip rebuilt)	from the look-ahead; if only the far part is seen, never above the last value and lower for a sharp curve	`true`	`false`
Line disappears in the **middle** of the image (glare, shadow, object over it) — up to `glareCoastMs` (600 ms)	**coasts**: the line's last motion continues, the heading part fades out	ramps down toward trackingMinValue	`false`	`true` (`coasting`)
Still hidden after the coast	±50 toward the side the line was last on	trackingMinValue (5)	`false`	`false`
Line leaves at the frame **edge** (sharp turn, robot drove off), or the look-ahead shows it leaving through the edge	±50 toward that edge, at once — never coasted	trackingMinValue (5)	`false`	`false`
… the same while glare hides the strip **and** the filter was following a curve toward that edge NEW	capped at the filter's own steering (same side) for up to ~0.17 s, then ±50	trackingMinValue (5)	`false`	`false`
With `glareMask = false`: line disappears mid-image — first `trackHoldFrames` frames (default 6)	**last value held** (v2)	**last value held**	`false`	`true`

SITUATION	STEERING RESULT[0]	TRACKING RESULT[1]	TRACKFOUND	LINEHELD
Line comes back inside the lock gate (where it left) within `trackLostGraceFrames` (default 8)	normal again — the **same** line is re-locked	normal	`true`	`false`
Gone for more than `trackLostGraceFrames`	stays at the last edge value (±50, slightly less if the calibrated column is far off-centre)	trackingMinValue	`false`	`false`
A new line appears after that **CHANGED**	full-width search: the band nearest the calibrated column with the calibrated width is taken — with corridorMode, the candidate that continues farthest up the look-ahead wins (also on the first frame after a coast)	normal	`true`	`false`

`FrameData.coasting` tells a coast from a v2 hold. Use `trackFound` (not `tracking == 5`) to show "TRACK LOST" — a hard turn also reads 5. With `trackMemory = false` there is no lock, no pinning and no hold: a missing line keeps the steering of its last position and tracking drops to the minimum. If calibration has not completed, see *If calibration cannot complete* above.

Runtime Utility Methods

```
int     fps       = pipeline.getFramesPerSecond();  // frames in the last completed second
long    frames    = pipeline.getTotalFrames();      // cumulative frame count since init
double  seconds   = pipeline.getElapsedSeconds();   // wall-clock seconds since first frame
boolean ready     = pipeline.isCalibrated();        // true once calibration is complete
boolean colours   = pipeline.isColorModelReady();   // true once colours are learned (or preset)
int     lineArgb  = pipeline.getLineColorArgb();    // learned line colour, 0xAARRGGBB
int     floorArgb = pipeline.getFloorColorArgb();   // learned floor colour, 0xAARRGGBB
boolean init      = pipeline.isInitialized();       // false before 1st frame / after release()
pipeline.requestRecalibration();  // re-learn colours + straight-ahead; then calibrate() again
pipeline.release();               // reset; the next frame re-initialises (calibrate again)
pipeline.init(960, 1280, cfg);    // optional: initialise now (size + config; drops calibration)
pipeline.setFrameCallback(cb);    // register overlay callback; pass null to unregister
```

Geometry getters (the same values the frame callback delivers — prefer the callback):

```
int     imgW    = pipeline.getImgW();                // frame size in pixels
int     imgH    = pipeline.getImgH();
int     stripTop = pipeline.getHRoiTop();            // scan-strip rows [top, bottom)
int     stripBot = pipeline.getHRoiBot();
int     vTop    = pipeline.getVRoiTop();             // far edge row of the look-ahead
int     vHalf   = pipeline.getVRoiHalfWidthPx();     // look-ahead half-width (config)
float   steerRef = pipeline.getSteerRefXFrac();      // calibrated straight-ahead columns
float   trackRef = pipeline.getTrackRefXFrac();
int     nPts    = pipeline.getLookAheadPointCount();  // dynamic road path (shared buffers)
float[] cols    = pipeline.getLookAheadColFracs();
float[] rows    = pipeline.getLookAheadRowFracs();
float   horizon = pipeline.getHorizonYFrac();        // floor horizon row from the mount tilt
float   scale   = pipeline.perspectiveWidthScale(0.5f); // relative apparent width at a row
```

Frame Callback — Overlay Geometry

Register a single callback once, and the library calls it automatically on **every calibration frame and every `processFrame()` frame**, passing a `FrameData` object with everything needed to draw a visual overlay and a status display: the H-ROI strip, the V-ROI look-ahead band, the dynamic road path, the live centroid, calibration progress and the learned colours.

No extra computation is added. The data already exists inside the pipeline — the callback just packages and delivers it (≈ 0.02 ms per frame). The callback runs on the thread that called calibrate / processFrame.

```
pipeline.setFrameCallback(data -> {
    // called automatically every frame
    // use data fields to draw your overlay
});

// To unregister:
pipeline.setFrameCallback(null);
```

Important: the same `FrameData` instance is reused on every call. Copy any fields you need to keep beyond the current frame.

FrameData Fields

FIELD	TYPE	DESCRIPTION
`imgW`, `imgH`	int	Frame size in pixels — the image all fractions refer to.
`hRoiTopFrac`	float	H-ROI top edge — fraction of frame height.
`hRoiBotFrac`	float	H-ROI bottom edge — fraction of frame height.
`vRoiTopFrac`	float	V-ROI far (top) edge — fraction of frame height.
`steerRefXFrac`	float	Calibrated straight-ahead column for steering — fraction of frame width.
`trackRefXFrac`	float	Calibrated straight-ahead column for look-ahead — fraction of frame width.
`halfBotFrac`	float	Near (bottom) half-width of the V-ROI box — fraction of frame width.
`halfTopFrac`	float	Far (top) half-width of the V-ROI box — fraction of frame width. Smaller than `halfBotFrac` due to perspective taper.
`ignoreTopFrac`	float	Top of the ignored robot-body band — fraction of frame height (`1` = nothing ignored).
`centroidXFrac`	float	Live detected line position — fraction of frame width (the real position, also during calibration). `-1` if the line was not found (also while it is held).
`trackWidthFrac`	float	Measured line width — fraction of frame width.
`tiltDeg`	float	Lean of the look-ahead line from vertical **in the image**, degrees (positive = leans right). For drawing only — the steering uses `headingDeg`. `0` before calibration or when the line is not found.
`lookAheadColFrac[]`	float[]	Column positions of the dynamic road poly-line, nearest to farthest. Array size **48**.
`lookAheadRowFrac[]`	float[]	Row positions of the dynamic road poly-line, nearest to farthest. Array size **48**.
`lookAheadPointCount`	int	Number of valid points in the path arrays. Only read up to this index.
`trackFound`	boolean	`true` when the line was detected in the H-ROI this frame. Use this for "TRACK LOST" — not `tracking > 5` (a hard turn also reads 5).
`isCalibrated`	boolean	`true` once calibration is complete.
`calibPhase`	int	`CALIB_LEARNING_COLORS`, `CALIB_FINDING_CENTER` or `CALIB_DONE`.
`calibProgress`	float	Overall calibration progress, `0 … 1`.
`calibIssue`	int	Current calibration problem (`ISSUE_*`, see *Calibration*).
`colorModelReady`	boolean	`true` when frames are processed in learned-colour mode.
`lineColorArgb`, `floorColorArgb`	int	Learned colours (`0xAARRGGBB`) — handy for UI swatches. `0` until learned.
`colorSeparation`	float	How well the two colours separate, in noise standard deviations.
`threshold`	int	Threshold used in the scan strip this frame; `-1` = no contrast (no line).
`steering`	int	Steering value, `-50 … +50` — same as `result[0]`. `0` during calibration.
`tracking`	int	Tracking value, `min … 100` — same as `result[1]`. `100` during calibration.
`steerOffsetTerm`	float	Offset (v1) part of the steering, in steering output units (`steerOffsetWeight` applied). `0` during calibration and when the line is not found.

FIELD	TYPE	DESCRIPTION
`steerHeadingTerm`	float	Heading part of the steering, in steering output units (already multiplied by `steerHeadingWeight` and `headingConfidence`). Before clamping and rounding, `steering ≈ steerOffsetTerm + steerHeadingTerm`. `0` during calibration and when the line is not found.
`headingDeg`	float	Line heading relative to the robot on the floor, perspective-corrected, minus the calibrated zero. Degrees, positive = the line heads to the right. `0` when unknown (line not found, too little look-ahead, or `steerHeadingWeight = 0`).
`headingConfidence`	float	Weight `0 … 1` the heading got this frame (`0` = too little / not a smooth line ahead → offset-only steering; also `0` when `steerHeadingWeight = 0`).
`lineHeld` CHANGED	boolean	`true` while the line is momentarily hidden mid-image: coasting (v3) or the last output held (`trackHoldFrames`, with `glareMask = false`).
`rejectedBands`	int	Number of candidate bands in the scan strip rejected or trimmed as distractors this frame (`0` with `rejectDistractors = false` and during calibration).
`baseReconstructed` NEW	boolean	`true` when the line was hidden in the scan strip (glare) and its position there was taken from the line farther ahead; `trackFound` is then `true`.
`coasting` NEW	boolean	`true` while the line is hidden mid-image and the pipeline coasts on its last motion (`lineHeld` is also `true`, `trackFound` is `false`).
`maskedFrac` NEW	float	Fraction `0 … 1` of the scan strip + look-ahead masked as glare (`0` with `glareMask = false`).
`corridorCoverage` NEW	float	Corridor tracker: fraction `0 … 1` of the near slices on the line this frame; `0` when the frame used the v2 logic (corner, dead end, re-acquiring, `corridorMode = false`).
`corridorFitRms` NEW	float	Corridor tracker: RMS distance of the slices from the fitted curve, in line widths.
`curveA`, `curveB`, `curveC` NEW	float	Corridor curve on the floor, `x = curveA + curveB·u + curveC·u²` (camera heights; `u` = distance beyond the scan strip, `x` = sideways, + = right). `0` when not used.

Drawing the overlay

Every field is a **fraction**, so convert to screen pixels first:

```
float screenX = data.centroidXFrac * screenWidth;   // any …XFrac field
float screenY = data.hRoiTopFrac   * screenHeight;  // any …YFrac / …TopFrac / …BotFrac field
```

Cropped previews: if the preview is scaled to fill the screen (Android `PreviewView` default FILL_CENTER), part of the image is cropped. Map with the same scale, or the overlay will drift: `scale = max(viewW / imgW, viewH / imgH)`, `x = (viewW − imgW·scale)/2 + xFrac·imgW·scale` (same for y). RoboPhoneApp's `OverlayView` does this.

H-ROI strip (a full-width horizontal band):

```
float top    = data.hRoiTopFrac * screenHeight;
float bottom = data.hRoiBotFrac * screenHeight;
canvas.drawRect(0, top, screenWidth, bottom, paint);
```

V-ROI trapezoid (perspective-tapered box — the far edge is narrower, and leans toward `trackRefX`):

```
float centre = data.steerRefXFrac * screenWidth;
float shear  = (data.trackRefXFrac - data.steerRefXFrac) * screenWidth;

// Near (wide) bottom corners — sit at the H-ROI top edge
float hTop = data.hRoiTopFrac * screenHeight;
float blX  = centre - data.halfBotFrac * screenWidth;
float brX  = centre + data.halfBotFrac * screenWidth;

// Far (narrow) top corners — shifted by shear so the box leans with the road
float vTop = data.vRoiTopFrac * screenHeight;
float tlX  = centre + shear - data.halfTopFrac * screenWidth;
float trX  = centre + shear + data.halfTopFrac * screenWidth;

// Connect the 4 corners:
//   bottom-left (blX, hTop) → bottom-right (brX, hTop)
//   → top-right (trX, vTop) → top-left (tlX, vTop) → close
```

Dynamic road band (bends through curves):

```
int n = data.lookAheadPointCount;
if (n >= 2) {
    for (int i = 0; i < n; i++) {
        float x = data.lookAheadColFrac[i] * screenWidth;
        float y = data.lookAheadRowFrac[i] * screenHeight;
        // connect points with a poly-line
        // i = 0 is nearest (bottom), i = n-1 is farthest (top)
    }
    // To draw as a band: trace the left edge (x - halfWidth) forward,
    // then the right edge (x + halfWidth) backward.
    // halfWidth tapers from halfBotFrac (near) to halfTopFrac (far).
}
```

Centroid indicator (live line position):

```
if (data.centroidXFrac >= 0) {
    float cx  = data.centroidXFrac * screenWidth;
    float top = data.vRoiTopFrac   * screenHeight;
    float bot = data.hRoiBotFrac   * screenHeight;
    // vertical line at cx, from top to bot
    // colour: green if data.trackFound, orange if not
}
```

State readout:

```
if (!data.isCalibrated) {
    status.setText("Calibrating " + Math.round(data.calibProgress * 100) + "%");
    // data.calibIssue explains a stall — see the Calibration table
} else if (data.lineHeld) {
    status.setText("Line hidden — holding S=" + data.steering);
} else if (!data.trackFound) {
    status.setText("Track lost");
} else {
    status.setText("Tracking  S=" + data.steering + "  T=" + data.tracking);
}
lineSwatch.setColor(data.lineColorArgb);   // learned colours
floorSwatch.setColor(data.floorColorArgb);
```

Steering debug readout (v2 fields — optional):

```
debug.setText(String.format("offset %.1f + heading %.1f (%.0f°, c=%.2f)  rejected=%d",
        data.steerOffsetTerm, data.steerHeadingTerm,
        data.headingDeg, data.headingConfidence, data.rejectedBands));
```

Full Example

```
LineFollowerPipeline pipeline = new LineFollowerPipeline();
final int W = 960, H = 1280;    // frame size (RoboPhoneApp: CameraX 1280 × 960, rotated to portrait)

// Optional tuning
LineFollowerPipeline.Config cfg = new LineFollowerPipeline.Config();
cfg.autoColor        = true;    // learn line + floor colours (any two colours)
cfg.cameraTiltDeg    = 45f;     // phone mount angle — measure it on your robot
cfg.bottomIgnoreFrac = 0.20f;   // hide robot body from scan
cfg.trackMemory      = true;    // lock onto line, ignore reflections
cfg.trackingMinValue = 5;       // never send a full stop
pipeline.configure(cfg);

// Register overlay callback — fires automatically on every frame
pipeline.setFrameCallback(data -> {
    myOverlay.drawHRoi(data.hRoiTopFrac, data.hRoiBotFrac);
    myOverlay.drawVRoi(data.vRoiTopFrac, data.steerRefXFrac,
                       data.trackRefXFrac, data.halfBotFrac, data.halfTopFrac);
    myOverlay.drawRoadPath(data.lookAheadColFrac,
                           data.lookAheadRowFrac,
                           data.lookAheadPointCount);
    myOverlay.drawCentroid(data.centroidXFrac, data.trackFound);

    if (data.isCalibrated) {    // drive only once calibration is complete
        robot.setSteeringAngle(data.steering);
        robot.setSpeed(data.tracking);
    }
});

// Phase 1 — Calibration (place robot correctly on the line first: straight, on a straight section)
Yuv f = camera.getYuvFrame();   // packed y / u / v planes
while (!pipeline.calibrateYuv420(f.y, f.u, f.v, W, H)) {
    f = camera.getYuvFrame();
}

// Phase 2 — Normal operation
while (running) {
    f = camera.getYuvFrame();
    pipeline.processFrameYuv420(f.y, f.u, f.v, W, H);
    // all results and overlay data delivered automatically via the callback
}
```

Behaviour Reference

SITUATION	STEERING	TRACKING
Straight line, centred	~0	~100
Line drifting to one side	± small	high
Curve approaching (line leaning)	varies	dropping toward the floor
Approaching a dead end / sharp corner	~0	ramps 100 → 5 with the remaining road
Robot at an angle to a straight line (line crossing the strip at the calibrated column)	≈ ±8 at 15°, ≈ ±17 at 30° *(heading part)*	high
Dark object / shadow beside or touching the line	unchanged *(object rejected or trimmed off)*	unchanged

SITUATION	STEERING	TRACKING
Line hidden for a moment in the middle of the image CHANGED	coasts for up to glareCoastMs *(v2: held trackHoldFrames)*	ramps down
Glare (lamp reflection) over the scan strip, line visible ahead NEW	follows the line ahead	from the look-ahead
Highlight on the centre of glossy tape NEW	unchanged *(both tape edges still seen)*	unchanged
Glare band across the look-ahead NEW	unchanged	unchanged *(not read as a dead end)*
Line running into a dark mat / ending beside a dark area NEW	~0	ramps 100 → 5 *(the mat's edge is not taken for the line)*
Blob of the line's colour touching the line NEW	unchanged	dips while the blob fills the far view, ≥ 60 once the line is seen past it
Hard turn	±50	5–30
Line completely lost	±50	5 *(the floor)*
Back on a straight after a turn	returns to 0	~100

Recommended Starting Values (lecturer's test program) CHANGED

For the Blockly test program — **motor steering = (0 − Steering) × c** and **motor power = round(x × (Power / 8) + 5)**, where *Steering* is `result[0]` and *Power* is the Tracking value `result[1]` — the closed-loop simulation (re-checked for v3, `_investigation/CHANGES_v3.md`) recommends the values below. **These are simulation results, to be confirmed on the robot in Stage B of** `ROBOT_TEST_CHECKLIST.md` (tests B1–B3).

SETTING	START WITH	NOTES
Steering gain **c**	`1.0` *(simulation result, to be confirmed in Stage B)*	Not `0.5`: the steering output was halved to ±50 (11 Oct 2026) after the program was written, so `0.5` now gives only half the steering authority. v3 simulation: `1.0` completed every run at realistic speed and every glare run; `0.5` is a gentler alternative on tracks without sharp corners. Keep `c ≤ 1.0` when power follows Tracking (`x = 1`).
Power mode **x**	`0` at our speed (~0.5–0.6 m/s); `1` only at low speed *(simulation result, to be confirmed in Stage B)*	With `x = 1` the power is about 6 … 18 and drops before turns. At about **0.5–0.6 m/s** the simulation weaves on straights with `x = 1` and `c = 1.0` (latency limit of the loop) — keep `x = 0` there, or use `c = 0.5` or a larger divisor (`Power / 12`) with `x = 1`.
Offset **+ 5**	`5`	If the robot does not move at power 5 (motor dead zone), raise it to 6–8 rather than raising `c`.
Library	defaults	v3 defaults (`corridorMode`, `glareMask`, `edgePairCheck` on). Set `cameraTiltDeg` to the real mount angle. If the robot cuts tight curves, lower `steerLookAhead` to `0.25`; if it hesitates when crossing the line at an angle, raise it to `0.75 … 1.0`.

These values come from simulation, not from the real robot: they are confirmed or changed by Stage B of `ROBOT_TEST_CHECKLIST.md` (keep `x = 0` if the measured Blockly loop rate is well below ~15 Hz).

Test Recording (v3) NEW

`LineFollowerPipeline.Recorder` records what the pipeline does, for tests on the phone. It is a `FrameCallback`: `attach` puts it in the pipeline's (single) callback slot and it forwards every frame to your own callback, so it needs nothing but the pipeline. The same code works in RoboPhoneApp and inside the RoboPhone system (a Blockly program through `LineFollowerHelper`). **Everything is off until switched on**; while off it only forwards the frame.

```
// once, after creating the pipeline (myOverlay = your own FrameCallback, or null)
LineFollowerPipeline.Recorder rec = LineFollowerPipeline.Recorder.attach(
        pipeline, context.getExternalFilesDir("linefollower"), myOverlay);
rec.setRecording(true);     // CSV log: one line per frame, a new file per start
rec.setCapture(true);       // keep the last ~2 s of frames in memory
rec.setAutoCapture(true);   // ... and save them on a coast, rebuilt base, rejected band, loss, jump
rec.captureNow();           // save the last ~2 s now (e.g. from a button)
rec.close();                // stop: write what is buffered, close the files (onDestroy)
```

WHAT	FILE	CONTENTS
CSV log (`setRecording`)	`lf_YYYYMMDD_HHMMSS.csv` + `.cfg.txt`	One line per `calibrate` / `processFrame` call: `t_ms` (time since the start), `frame`, `processing` (0 = calibration frame), every scalar `FrameData` field (look-ahead path: first and last point), `fps` and `lib_ms` (time inside `processFrame`). The `.cfg.txt` holds the Config in use.
Capture (`setCapture` + `captureNow` / `setAutoCapture`)	`lfcap_YYYYMMDD_HHMMSS_<reason>.bin`	The last `captureFrames` (120) frames downscaled by `captureScale` (4), the calibration frames, the phone's steering / tracking / flags per frame and the Config. `<reason>`: `manual`, `coast`, `recon`, `reject`, `lost`, `jump`.

Recorder Settings

Set these before switching capture on.

FIELD	TYPE	DEFAULT	DESCRIPTION
`captureFrames`	int	120	Frames kept for a capture (~2 s at 60 fps). Set before `setCapture(true)`.
`captureScale`	int	4	Downscale factor of the saved frames (`1` = full size).
`postTriggerFrames`	int	30	Auto-capture: frames still recorded after the event before saving (~0.5 s at 60 fps).
`maxCaptures`	int	10	Most captures one recorder saves.
`jumpTrigger`	int	30	Auto-capture: a steering change of at least this much between two frames is an event.

- **Cost.** The camera thread does not allocate and never waits for the disk; a background thread writes the files. If that thread falls behind, CSV lines are dropped and counted (`getDroppedLines()`). Desktop, 1280 × 720 YUV: CSV log ≈ 0 ms per frame; capture ≈ +0.2 ms per frame, about 20 MB of memory at 960 × 1280; a saved capture is about 15 MB.
- **Replay on the desktop.** `gradlew replay -Pcapture=<lfcap_….bin>` in `LineFollowerLibrary/` scales the frames back up, calibrates on the saved calibration frames, runs the saved frames and prints the phone's steering next to the replay's. `-Pset=glareMask=false,corridorMode=false` shows the same scene with the v2 logic. The replay is close, not bit-exact (downscaled frames; the window starts without the phone's earlier state). Captures copied to `LineFollowerLibrary/src/test/captures/` are replayed by `gradlew test`.
- **Summary of a CSV.** `py tools/analyze_recording.py <file or folder> [--from s --to s]` prints fps and `lib_ms`; steering (largest step, jumps ≥ 25, frames at ±50 and how many had glare in view); tracking; and the coast, hold, rebuilt-base and rejected-band spells with their times.

- **Getting the files off the phone.** `adb pull /sdcard/Android/data/<app package>/files/linefollower` (RoboPhoneApp: `com.example.firstproject`), or Android Studio's Device Explorer. Uninstalling the app deletes the folder. Details for both apps: `ROBOT_TEST_CHECKLIST.md` §A0.4 and §B0.

Notes

- The pipeline is **not thread-safe**. Call `calibrate()` / `processFrame()` (and `requestRecalibration()`) from a single thread only — e.g. have a UI button set a flag that the camera thread checks.
- The returned `int[]` is reused on every call. Copy the values if you need them past the next frame.
- Calibration assumes the robot is correctly placed on the line, with floor visible on both sides. If it is not, call `requestRecalibration()` once it is repositioned, then calibrate again. Recalibrate whenever you move to a track with different colours, different lighting or a different tape width.
- A line that jumps sideways by more than the line-lock gate in a single frame (`trackMemoryGateMult` × line width, around the line's predicted position) is treated as leaving the line — this is what makes the lock ignore reflections.
- The heading part of the steering relies on `cameraTiltDeg` and `verticalFovDeg` matching the real camera. The Android app runs at 960 × 1280 (CameraX picks 1280 × 960, rotated to portrait).
- Very harsh scenes can still defeat colour learning: e.g. a dark floor in deep shadow, with strong texture, where the tape is only ~10 grey-levels darker than the shadowed floor. Even lighting helps.
- Performance: ≈ 0.6 ms per frame in YUV colour mode at 960 × 1280 on a desktop CPU with the v3 defaults (v2: ≈ 1.0 ms — the corridor tracker converts only the image rows it reads). A phone is several times slower, still far below the 16.7 ms of 60 Hz. All per-frame buffers are pre-allocated (no garbage per frame, in all three input formats — checked by a unit test).
- The coast clock uses `System.nanoTime()`; when replaying recorded frames or simulating, set `frameIntervalMs` to the real frame interval.

RoboPhoneApp — Android Reference Integration

`RoboPhoneApp/` runs the library live on the phone camera (CameraX → packed YUV 4:2:0 → `calibrateYuv420` / `processFrameYuv420`). Everything on screen comes from the `FrameCallback`:

- **Overlay** — cyan STEER strip with the detected line, magenta look-ahead band that bends along the line, dashed calibrated straight-ahead line, red ignored robot-body band; mapped with the preview's FILL_CENTER crop so it lines up with the camera image.
- **HUD** — STEER bar (−50…+50), TRACK bar (min…100 %), status (calibration progress and problems / TRACKING / TRACK LOST), FPS with the time per frame in the pipeline and for the whole analyzer, learned LINE and FLOOR colour swatches, threshold and colour separation.
- **RECALIBRATE** button — re-learns the colours and the straight-ahead reference (place the robot on the line first).
- **v3 test settings** NEW (all off by default, remembered between runs): **AE/AWB LOCK** locks auto-exposure and white balance when calibration completes (Camera2Interop; RECALIBRATE unlocks first), **EV** sets exposure compensation, **v3 HUD** shows corridor coverage / fit, COAST / RECON, the masked fraction, rejected bands, the lock state and EV, and a second line with the largest steering step, the jumps ≥ 25 and the frames at ±50 (tap to reset). With all off the camera is set up exactly as before.
- **Test recording** NEW (off at every start): **REC CSV** writes the per-frame CSV log, **CAPTURE** keeps the last ~2 s of frames and saves them on events, **SAVE 2s** saves them now. Files: `/sdcard/Android/data/com.example.firstproject/files/linefollower/`.
- **Logcat** (`adb logcat -s RoboPhone`): every 30 frames `PERF fps=… pipe_ms=… pipe_max_ms=… frame_ms=…`; `REC off: <file>`, `CAPTURE saved <file>`. The tests are in `ROBOT_TEST_CHECKLIST.md` (Stage A: by hand with this app; Stage B: closed loop inside RoboPhone).

The app builds the library **from source** through a Gradle composite build (`includeBuild("../LineFollowerLibrary")`), so library changes show up in the app on the next build — no jar copying. Library tests: `cd LineFollowerLibrary && gradlew test` (format / colour unit tests plus camera-simulator scenes: shadows, wood grain, turns, line loss, distractor tapes, dead ends).

Tests: all **50 tests pass** with the v3 defaults (one more replays phone captures and is skipped until some are added): 24 unit tests (formats, colours, calibration, steering parts, the v2 hold with `glareMask = false`, the v3 coast clock, v1 behaviour), 6 camera-simulator scenes, 13 v3 scenarios at 0.6 m/s / 30 fps in gray and YUV (glare on the strip, across the look-ahead and on the tape centre, sharp curve and edge exit under glare, objects and blobs beside the line, re-acquiring, a line running into a mat, a 90° corner and a dead end) a zero-allocation test, and 6 tests of the test recorder (transparent when off, CSV contents, Config round trip, desktop replay of a capture, auto-capture, zero allocations on the camera thread with recording on). `gradlew perf` times v3 against the v2 settings; `gradlew checkJava8` checks the library compiles as Java 8; `gradlew replay -Pcapture=…` replays a capture from the phone. `gradlew jar` and the app build do not run the tests.

Robot Results (v3) — pending NEW

v3 has been tested in simulation only (unit scenarios and the closed-loop simulator, `_investigation/CHANGES_v3.md`). The tests on the phone run in two stages, described in `ROBOT_TEST_CHECKLIST.md`. **Stage A** is by hand with RoboPhoneApp: glare on the tape and on the floor, objects beside and touching the line, a 90° corner, an exit off the line, AE/AWB lock off / on, and PERF. **Stage B** is the closed loop inside RoboPhone: weaving, the Blockly values `c` and `x`, ~0.6 m/s, and full laps. Both stages record CSV logs and captures. The results, and any default changes they lead to, will be added here.